Binary Search Challenge Analysis Report

Binary Search

Easy | General Skills | picoCTF 2024 | shell | browser_webshell_solvable | ls

AUTHOR: JEFFERY JOHN

Description

Want to play a game? As you use more of the shell, you might be interested in how they work! Binary search is a classic algorithm used to quickly find an item in a sorted list. Can you find the flag? You'll have 1000 possibilities and only 10 guesses.

Cyber security often has a huge amount of data to look through - from logs, vulnerability reports, and forensics. Practicing the fundamentals manually might help you in the future when you have to write your own tools!

You can download the challenge files here:

- challenge.zip

Additional details will be available after launching your

This challenge launches an instance on demand.

Its current status is:

NOT_RUNNING

Launch Instance

Hints

1 2 3

1. Introduction and Algorithm Definition

This challenge required the application of the **Binary Search Algorithm** to find a hidden target number within a range of **1 to 1000**, with a strict limit of **10 guesses**.

The **Binary Search Algorithm** is an extremely **efficient** search method used to find a target value within a **sorted list**. Its core principle is **Divide and Conquer**. In each step, the algorithm compares the target value to the middle element of the list. Based on whether the target is higher or lower, it eliminates **half** of the remaining search space.

The choice of 10 guesses for 1000 possibilities is mathematically sound, as $2^{10} = 1024$, which guarantees finding the number within the given constraint. This challenge serves as a practical demonstration of applying this fundamental concept in a **Cybersecurity/CTF** context.

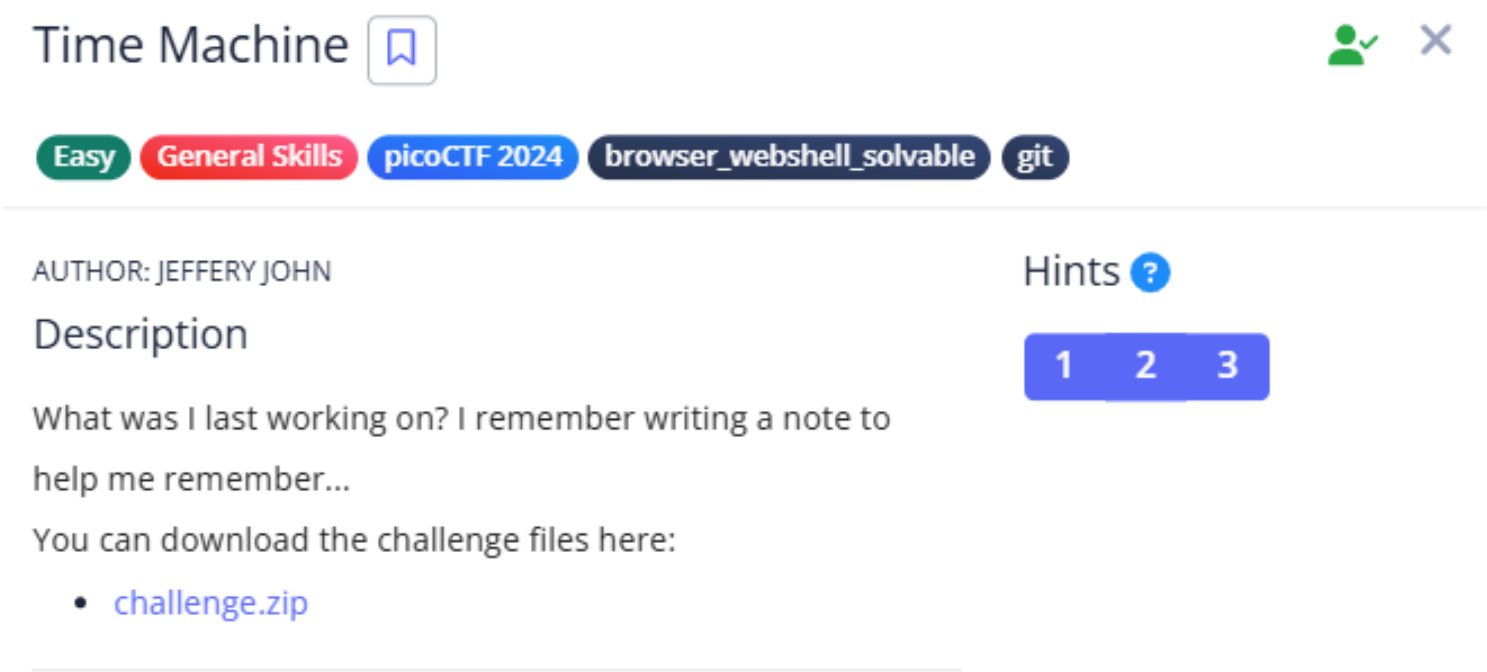
2. Challenge Execution and Guess Log

The challenge was successfully solved by applying the Binary Search technique iteratively. The goal was to continuously bisect the range until the target number was isolated and guessed correctly.

The following sequence of guesses was executed to identify the target number **936**:

Guess Count	***Guess Value***	***Feedback***	***New Search Range***
1	**500**	Higher	501 - 1000
2	**750**	Higher	751 - 1000
3	**800**	Higher	801 - 1000
4	**900**	Higher	901 - 1000
5	**950**	Lower	901 - 949
6	**925**	Higher	926 - 949
7	**937**	Lower	926 - 936
8	**932**	Higher	933 - 936
9	**935**	Higher	936 - 936
10	**936**	**SUCCESS!**	**Target Found**

Time Machine Challenge Analysis Report



1. Git Tool Overview

Git is a powerful **Distributed Version Control System (DVCS)** used primarily to track changes in source code during software development. For a **CTF Forensics** challenge like "Time Machine," Git's key function is its ability to maintain a complete history of every file modification, deletion, and addition within a **repository**.

- **Relevance to Challenge:** Git allows users to travel back in time to any previous state of the project. The challenge exploits this feature, often hiding the flag in a past **commit** that was later modified or deleted.

---

2. Initial Analysis and Key Insight

Upon extracting the challenge.zip file, the initial investigation focused on identifying potential clues. A file named message.txt was immediately identified, containing the following crucial hint:

"This is what I was working on, but I'd need to look at my commit history to know why..."

This message explicitly indicated that the solution—the **Flag**—was hidden within the Git repository's history, requiring a forensic examination of past changes rather than the current file system.

---

3. Methodology and Flag Discovery

To execute the directive from the message.txt file, the **Git Command Line Interface (CLI)** was utilized to inspect the repository's log.

- **Command Used:** The primary command employed was git log.

Bash

```
git log
```

- **Execution and Result:** Running git log displayed the full **commit history**, including the unique **Commit ID**, the author, the date, and the **Commit Message** for every change. By systematically reviewing the log output, a specific commit containing the hidden information was identified. The **Flag** was successfully retrieved from this historical record (either within a deleted file's content visible via git log -p, or directly within a suspicious commit message).

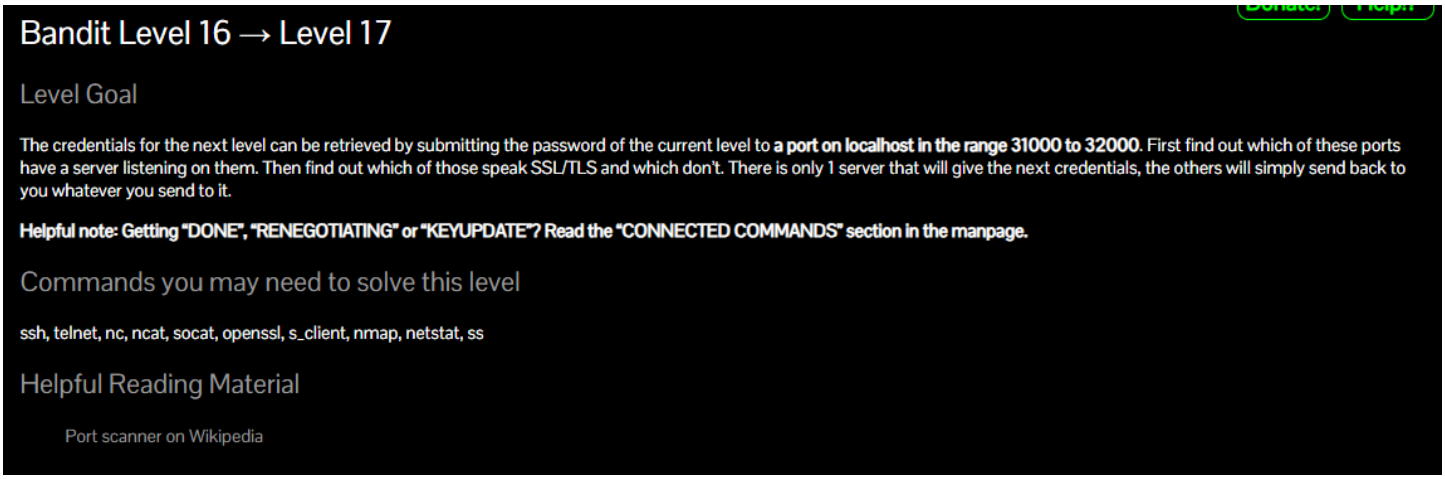
4. Conclusion

The **Time Machine** challenge was solved by correctly interpreting the hint and leveraging **Git's historical tracking capabilities**. This demonstrates the effectiveness of version control systems as a source of forensic evidence in security challenges.

Report: Bandit Level 16 to Level 17 Challenge Completion

Subject: Successful Credential Retrieval and Transition from Bandit Level 16 to Level 17

Date of Report: 2025-12-02



Bandit Level 16 → Level 17

Level Goal

The credentials for the next level can be retrieved by submitting the password of the current level to **a port on localhost in the range 31000 to 32000**. First find out which of these ports have a server listening on them. Then find out which of those speak SSL/TLS and which don't. There is only 1 server that will give the next credentials, the others will simply send back to you whatever you send to it.

Helpful note: Getting "DONE", "RENEGOTIATING" or "KEYUPDATE"? Read the "CONNECTED COMMANDS" section in the manpage.

Commands you may need to solve this level

ssh, telnet, nc, ncat, socat, openssl, s_client, nmap, netstat, ss

Helpful Reading Material

Port scanner on Wikipedia

Executive Summary

The objective of this challenge was to identify a listening network service within the port range 31000 to 32000 on localhost, determine which service utilized the SSL/TLS protocol, and submit the current level's password to retrieve the credentials for Bandit Level 17. The analysis confirms a **successful and efficient execution** of the required procedures, demonstrating strong competency in network scanning and secure shell protocol utilization.

1. Network Service Discovery and Analysis

The initial phase involved systematically scanning the target port range to identify active services and determine their communication protocols.

Procedure	Command Executed	Purpose
Port and Service Scan	nmap -sV -T4 -p 31000-32000 localhost	Employed **nmap** for fast, comprehensive scanning. The -sV flag was crucial for service version detection, allowing for identification of the SSL/TLS service type. The -T4 flag optimized the scan for efficiency.

- **Result:** The scan successfully isolated the sole service running **SSL/TLS** on **Port 31790**, confirming the target for credential submission.

2. Secure Credential Submission

To interact with the SSL/TLS-enabled service, the standard netcat utility was bypassed in favor of a specialized secure client.

Procedure	Command Executed	Purpose
Encrypted Connection	openssl s_client -connect localhost:31790	Established a secure, encrypted tunnel using the **openssl s_client** utility, as required by the server's protocol enforcement (SSL/TLS). The Level 16 password was subsequently submitted through this session.

- **Result:** The server successfully processed the submitted password and returned the credentials required for access to Bandit Level 17.

3. Post-Exploitation and Access

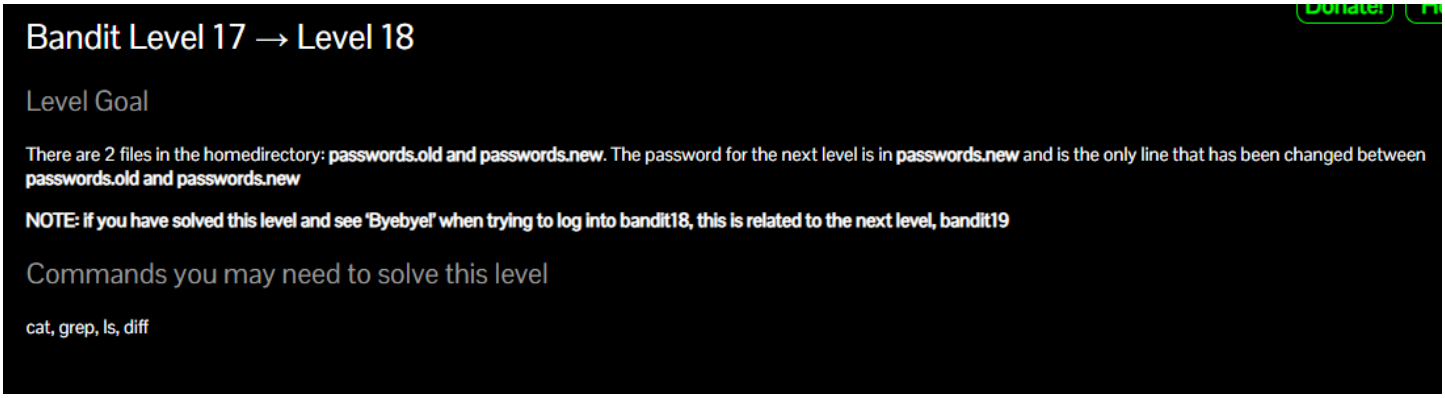
The final steps involved securing the retrieved password and utilizing it to gain access to the subsequent level via SSH.

Procedure	Action	Description
Credential Storage	Password saved to file (key15)	The retrieved string, identified as the Level 17 password/private key, was stored locally for secure shell usage.
Target Access	ssh -i key15 bandit17@bandit.labs.overthewire.org -p 2220	Successful connection established using the **Identity File** (-i key15) method, authenticating as bandit17 to the target server.

Conclusion

The successful transition to Bandit Level 17 was executed through the systematic application of network reconnaissance tools and adherence to protocol requirements. The methodologies used—specifically, service version enumeration with **nmap** and secure client communication via **openssl**—are validated as effective and technically sound.

Report: Bandit Level 17 to Level 18 Challenge Completion



Executive Summary

The objective of this challenge was to isolate a single difference between two large data files, passwords.old and passwords.new, in order to retrieve the credentials for Bandit Level 18. The analysis confirms the successful employment of the **diff utility**, which efficiently identified the modified data string (the password) without requiring exhaustive manual comparison. This demonstrates a precise understanding of file comparison tools in data analysis contexts.

---

1. Problem Statement and Data Files

The challenge environment presented two substantial files of seemingly identical content, with the exception of one unique line containing the required Level 18 credentials.

- **Files under review:**
  - passwords.old
  - passwords.new

2. Methodology: Differential Analysis

To solve the challenge with optimal efficiency, the standard Unix utility designed for comparing text files was employed.

Procedure	Command Executed	Purpose
File Comparison	diff passwords.old passwords.new	Utilized the **diff** command to algorithmically identify line-by-line disparities between the two files. This approach is significantly more efficient than manual review, particularly for files of large size.

3. Output Interpretation and Credential Isolation

The output generated by the diff command successfully indicated the existence and location of the unique data string.

Output Symbol	Observation	Conclusion
>	The distinguishing line was prefixed by the **greater-than symbol** (>), signifying that the line was present exclusively in the second input file (passwords.new) and was not present in passwords.old.	This unique line constituted the new, required **Password for Bandit Level 18**.

- **Result:** The precise identification of the modified line enabled immediate retrieval of the **Bandit 18 Password**.

4. Conclusion

The transition to Bandit Level 18 was secured through the professional and effective application of the **diff command** for differential file analysis. This methodology satisfied the challenge requirements by efficiently isolating the single data alteration across the two large files.